

I use AI tools for the following tasks,

AI Critique

During this assignment, working with AI to create test documents showed both its strengths and weaknesses. AI was very helpful in organizing information and creating test cases quickly, but it also missed some important points.

One major issue was that the AI did not identify some boundary conditions. For example, it did not consider maximum lengths for user names, passwords, or the maximum number of items in a cart. Because these limits were not clearly stated in the requirements, the AI only followed the given information and did not think about possible system limits. As a result, some edge cases could be missed, such as UI problems caused by entering an extremely long user name.

Another weakness was that the AI sometimes assumed that testing at different system layers was unnecessary. For example, it treated password confirmation and input validation as frontend features only and did not consider what might happen if a user bypassed the frontend and sent requests directly to the backend. This happened because the AI generally assumes that the system works as intended unless negative scenarios are clearly requested.

The most important lesson from this experience is that AI should be used as a supporting tool rather than a final source of truth. AI is useful for creating structured documents and converting requirements into test cases, but human testers are still responsible for finding hidden risks, thinking about unusual user behavior, checking database integrity, and identifying security issues such as duplicate email validation or unauthorized data changes. Human judgment is necessary to cover the gaps that AI may overlook.

AI Audit Record

Metadata

Field	Value
Feature	Agent Skill – AI Audit Appendix Generator
Tool	ChatGPT
Timestamp	03/07/2026 12:02

Original Prompt (Verbatim)

Tôi cần chỉnh sửa lại phần AI Audit Skill này, hiện tại output chưa ra đúng, AI vẫn đang rút gọn output nên tôi muốn điều chỉnh lại.

Hãy cho tôi bản đã điều chỉnh skill này.

AI Output (Verbatim)

The skill was redesigned to:

- Remove automatic session scanning.
- Preserve prompts and outputs verbatim.
- Prevent AI self-evaluation.
- Prevent fabrication of metadata.
- Add handling for long outputs.
- Reserve all verdict and correction sections for human review.

Generated artifact:

SKILL: AI Audit Record Generator (HW02 Compliant)

Main sections:

- Purpose
- Core Compliance Rules
- Verbatim Preservation
- No Self-Evaluation
- No Fabricated Metadata
- Human Ownership
- Long Output Handling
- Expected Input Format
- Required Output Template
- Trigger Commands

Generated Artifact

SKILL.md

Purpose:

Standardize AI audit evidence collection for HW02 while ensuring that prompts and outputs are preserved exactly and that all evaluation remains the responsibility of the student.

Student Notes

The initial version of the skill attempted to automatically extract content from chat history and occasionally produced shortened outputs.

The revised version was created to enforce verbatim preservation,

prevent AI-generated verdicts, and improve compliance with HW02 AI audit requirements.

Usage Outcome

The generated skill was stored as:

.github/skills/AI_AUDIT/SKILL.md

and will be used to generate standardized AI Audit entries throughout the HW02 workflow.

AI Audit Record

Metadata

Field	Value
Feature	FR-01 Web Registration
Tool	GitHub Copilot Chat
Timestamp	04/07/2026 10:37

Original Prompt (Verbatim)

Apply Domain Testing for FR-01 Account Registration and structure the result into step-by-step explanation plus a test case table.

AI Output (Verbatim)

* **Input Variables & Constraints:** Full Name, Email, Password, Confirm Password; constraints: Full Name required; Email must be valid and unique; Password must be strong with Min length = 8, >=1 uppercase, >=1 lowercase, >=1 digit, >=1 special char from allowed set (@, \$, !, %, *, ?, &); Confirm Password must match Password; success redirects to Login.
* **Equivalence Partitions / Boundaries Identified:**
 * *Partition N-V1:* Full Name valid non-empty value, representative Nguyen Van A.
 * *Partition N-I1:* Full Name empty/whitespace, representative empty string.
 * *Partition E-V1:* Email valid and unique, representative newuser01@eshop.com.
 * *Partition E-I1:* Email format invalid, representative newuser01@eshop.
 * *Partition E-I2:* Email already exists, representative test@eshop.com.

```
* *Partition P-V1:* Password meets all strength rules, representative
Password123!.
* *Partition P-I1:* Password length < 8, representative Abc1!d.
* *Partition P-I2:* Password missing uppercase, representative password123!.
* *Partition P-I3:* Password missing lowercase, representative PASSWORD123!.
* *Partition P-I4:* Password missing digit, representative Password!!!.
* *Partition P-I5:* Password missing special character, representative
Password1234.
* *Partition C-V1:* Confirm Password matches Password exactly, representative
Password123!.
* *Partition C-I1:* Confirm Password mismatch, representative Password124!.
* **Generated Test Cases Structural Summary:** TC-01 (Happy Path), TC-02 (Blank
Full Name), TC-03 (Invalid Email Format), TC-04 (Duplicate Email), TC-05 (Password
Too Short), TC-06 (No Uppercase), TC-07 (No Lowercase), TC-08 (No Digit), TC-09
(No Special Character), TC-10 (Confirm Password Mismatch).
```

AI-002 – Skill Refinement and Compliance Improvement

Metadata

Field	Value
Feature	Agent Skill – Domain & BVA Testing
Tool	GitHub Copilot Chat
Timestamp	05/07/2026 08:10

Purpose

Improve the AI Audit Skill after observing that generated audit records occasionally summarized or truncated AI outputs.

Interaction Summary

The student reviewed the initial version of the AI Audit Skill and identified the following issues:

- AI outputs were sometimes shortened.
- Verbatim preservation was not guaranteed.
- The skill relied on automatic session extraction.
- The skill could encourage AI-generated evaluations.

The student requested improvements to strengthen compliance with HW02 AI audit requirements.

AI Contribution

The AI proposed:

- Explicit verbatim-preservation rules.
- Prohibition of AI-generated verdicts.
- Human-owned review sections.
- Long-output handling procedures.

- A standardized audit record template.

Human Review

The student reviewed the proposal and selected the following changes:

- Remove automatic session scanning.
- Require manual Prompt/Output input.
- Keep verdict and correction sections empty.
- Add compliance checklist items.
- Strengthen anti-fabrication requirements.

Resulting Artifact

Generated file:

`.github/skills/Al_AUDIT/SKILL.md`

Purpose:

Provide a reusable workflow for documenting AI interactions while ensuring that final evaluation remains the responsibility of the student.

Outcome

The revised skill was adopted for subsequent AI audit activities in HW02.

AI Audit Record

Metadata

Field	Value
Feature	FR-06 Mobile Product Detail
Tool	GitHub Copilot Chat
Timestamp	05/07/2026 10:50

Original Prompt (Verbatim)

Apply test design: Mobile - FR-06

AI Output (Verbatim)

FR-06 Mobile - Xem chi tiết sản phẩm

Stage A - Domain variables

A1. Input variables

Layer	Variable	Domain / meaning	Constraint source
UI	Product selection	product id	Product selected from the mobile product list before opening the detail screen FR-06, FR-05
UI	Quantity text input	User-entered quantity on the product detail screen	FR-06
UI	Add-to-cart action	Tap on the add button after the detail screen is loaded	FR-06
API	`id` path parameter in `GET /api/products/:id`	Product identifier used to fetch detail data	API spec 3.2
API	Request body in `POST /api/cart`	Payload used to add an item to the server cart	API spec 4.2
API	`Authorization` header	Token required for cart API access	API spec 4.1 and backend route guard

A2. System state variables

Layer	State variable	Domain / meaning	Why it matters
UI	Current view	Home, product detail, or cart	Controls whether the user can inspect detail data and add to cart
UI	Product fetch result	Loaded product, empty object, or network/error state	Determines whether the detail screen renders the product or a fallback state
UI	Cart contents	Empty cart, cart with one item, cart with same item already present	Needed to verify add-to-cart behavior and quantity aggregation
API	Product existence	Existing row vs missing row in `products` table	Controls the response of `GET /api/products/:id`
API	Authentication state	Valid token vs missing/invalid token	Required for `POST /api/cart`
API	Cart storage state	Existing server cart array vs new server cart array	Affects duplicate add behavior on the backend

A3. Output variables

Layer	Output	Expected shape
UI	Product detail screen	Image, name, price, description, category information, quantity field, add button
UI	Feedback after add	Toast, alert, or badge/cart update that confirms the add action
UI	Cart change	New line item or incremented quantity for the same product
API	Product detail response	JSON object for an existing product, or graceful not-found style response for missing product
API	Cart response	Success message or validation/auth error with correct HTTP status

A4. Cross-variable constraints

Constraint ID	Constraint
---	---

```
| C-01 | Quantity must be a positive integer and must be at least 1 |
| C-02 | The add-to-cart action must use the selected product and the current
quantity value together |
| C-03 | Opening the detail screen must not mutate cart contents |
| C-04 | Adding the same product again must increment its quantity instead of
creating an unrelated product entry |
| C-05 | `GET /api/products/:id` must only read product data and must not change
state |
| C-06 | `POST /api/cart` must require a valid token and a complete payload |
```

Stage B - Equivalence partitions

```
| Partition ID | Description | Valid or invalid | Representative value |
Requirement basis | Risk | Type (Specification-derived or Exploratory) |
|---|---|---|---|---|---|---|
| UI-P-01 | Existing product selected from the list and detail data loads normally
| Valid | Product id `1` | FR-06 detail view | Core happy path may fail if
navigation or fetch is broken | Specification-derived |
| UI-P-02 | Quantity is a positive integer | Valid | `1` | FR-06 quantity minimum
| Add-to-cart must accept the only explicit valid lower bound | Specification-
derived |
| UI-P-03 | Same product already exists in cart | Valid | Product id `1` already
in cart | FR-06 plus cart consistency | Quantity aggregation may be implemented
incorrectly | Specification-derived |
| UI-P-04 | Product does not exist or fetch returns an empty object | Invalid |
Product id `999999` | System-level handling of missing product data | App may
crash or show a blank screen | Exploratory |
| UI-P-05 | Quantity is `0` | Invalid | `0` | FR-06 minimum boundary | Quantity
below the minimum must not be accepted | Specification-derived |
| UI-P-06 | Quantity is negative | Invalid | `-1` | FR-06 positive integer rule |
Negative quantity may be normalized or mishandled | Specification-derived |
| UI-P-07 | Quantity is a non-integer value | Invalid | `1.5` | Positive integer
rule | Decimal input may bypass weak parsing | Exploratory |
| UI-P-08 | Quantity is non-numeric text or blank | Invalid | `abc` / empty string
| Positive integer rule | Text input can reach the parser through the mobile
keyboard | Exploratory |
| API-P-01 | `GET /api/products/:id` with an existing id returns product data |
Valid | `GET /api/products/1` | API spec 3.2 | Failure blocks the detail screen |
Specification-derived |
| API-P-02 | `GET /api/products/:id` with a missing or malformed id | Invalid |
`GET /api/products/0` | Product id must refer to an existing row | Missing product
handling may be inconsistent | Specification-derived |
| API-P-03 | `POST /api/cart` with valid token and complete item body | Valid |
`{id:1,name:"...",price:30000000,quantity:1}` | API spec 4.2 | Core server-side
cart path may fail | Specification-derived |
| API-P-04 | `POST /api/cart` without Authorization header | Invalid | No token |
API spec 4.1 and backend authentication | Security boundary may be bypassed |
Specification-derived |
| API-P-05 | `POST /api/cart` with partial or tampered JSON body | Invalid |
Missing `quantity` or `price` | Bypass UI validation directly to backend | Backend
may accept malformed data | Exploratory |
| API-P-06 | `POST /api/cart` with invalid quantity value | Invalid | `quantity:
0` or `-2` | Positive integer rule | Server-side validation may be missing |
Specification-derived |
```

Stage C - Boundary analysis

Boundary	Just below	At boundary	Just above
Quantity minimum	`0`	`1`	`2`
Quantity sign / integer rule	`-1` or `1.5`	`1`	`2`
Product id access boundary, using the lowest seeded row as the minimum existing id	`0`	`1`	`2`
Cart payload completeness boundary	Missing required field such as `quantity`		
Full required body: `id`, `name`, `price`, `quantity`	Full body plus an extra unrelated field		

Notes:

- The quantity boundary is specification-derived because FR-06 explicitly states that quantity must be a positive integer with a minimum of 1.
- The product-id boundary is an implicit system boundary derived from the database's autoincrement primary key and the seeded dataset.
- The cart payload boundary is a system-level schema boundary for bypassing the mobile UI and testing the backend directly.

Stage D - Combination strategy

The UI test set uses one-factor-at-a-time around a single valid baseline: a real product opens correctly, quantity is `1`, and the add button is tapped once. From that baseline, each negative case changes only one driver at a time: product existence, quantity class, or cart state. This avoids a full Cartesian product of product data, quantity values, and screen states.

The API test set uses a small decision table instead of combining every product, quantity, and token combination. The key decisions are: product exists or not, token present or missing, and payload complete or tampered. That gives coverage of the server-side boundary conditions without multiplying equivalent cases that would exercise the same code path.

Stage E - Test cases

TC ID	Feature	Platform	Layer (UI / API)	Technique	Requirement ID	Objective	Preconditions	Test Data	Steps	Expected Result	Actual Result	Status (NOT RUN)	Evidence	AI origin (AI Generated)	Human revision	Notes
FR06-UI-01	FR-06 Mobile Product Detail	Mobile	UI	Domain Testing	FR-06	Verify that a valid product opens the detail screen with all required fields visible	Mobile app is running; product list is loaded; product id `1` exists	Product id `1`	1. Open the product list. 2. Tap a valid product. 3. Observe the detail screen.	The app shows the product image, name, price, description, and category information without crashing.		NOT RUN		AI Generated		
FR06-UI-02	FR-06 Mobile Product Detail	Mobile	UI	Domain Testing	FR-06	Verify that the quantity field starts at the minimum valid value	Detail screen for an existing product is open	Default quantity state	1. Open a product detail screen. 2. Observe the quantity field before editing.	The quantity field is initialized to `1`.		NOT RUN		AI Generated		
FR06-UI-03	FR-06 Mobile Product Detail	Mobile	UI	Boundary Value Analysis	FR-06	Verify the exact minimum valid quantity	Detail screen for an existing									

product is open | Quantity `1` | 1. Enter `1` in the quantity field. 2. Tap Add to cart. | The product is added successfully and the confirmation feedback is shown. | | NOT RUN | | AI Generated | | |

| FR06-UI-04 | FR-06 Mobile Product Detail | Mobile | UI | Boundary Value Analysis | FR-06 | Verify that quantity `0` is not accepted | Detail screen for an existing product is open | Quantity `0` | 1. Enter `0` in the quantity field. 2. Tap Add to cart. | The system does not accept a quantity below 1; it either blocks the add action or normalizes the value to the minimum valid quantity. | | NOT RUN | | AI Generated | | |

| FR06-UI-05 | FR-06 Mobile Product Detail | Mobile | UI | Boundary Value Analysis | FR-06 | Verify that negative quantity is not accepted | Detail screen for an existing product is open | Quantity `-1` | 1. Enter `-1` in the quantity field. 2. Tap Add to cart. | The system does not accept the value as a valid quantity below 1. | | NOT RUN | | AI Generated | | |

| FR06-UI-06 | FR-06 Mobile Product Detail | Mobile | UI | Exploratory Robustness Test | FR-06 | Verify that non-integer input does not bypass the positive-integer rule | Detail screen for an existing product is open | Quantity `1.5` or `abc` | 1. Enter a decimal or text value. 2. Tap Add to cart. | The system does not add the item with a non-integer quantity; it should keep the quantity valid or show validation feedback. | | NOT RUN | | AI Generated | | |

| FR06-UI-07 | FR-06 Mobile Product Detail | Mobile | UI | Domain Testing | FR-06 | Verify that adding the same product twice updates the existing cart entry | Same product is already present in the cart | Product id `1`, quantity `1` then `1` again | 1. Open the same product detail twice. 2. Add it to cart each time. 3. Check the cart contents. | The cart keeps one line item for that product and increases its quantity. | | NOT RUN | | AI Generated | | |

| FR06-UI-08 | FR-06 Mobile Product Detail | Mobile | UI | Exploratory Robustness Test | FR-06 | Verify graceful handling of a missing product | Mobile app can request a non-existent id | Product id `999999` | 1. Open a product detail link for a missing id. 2. Observe the screen state. | The app shows a graceful empty/not-found state and does not crash. | | NOT RUN | | AI Generated | | |

| FR06-API-01 | FR-06 Mobile Product Detail | Mobile | API | Domain Testing | FR-06 / API 3.2 | Verify that a valid product id returns a complete detail payload | Backend API is running; product id `1` exists | `GET /api/products/1` | 1. Send the request directly to the backend. 2. Inspect the response body. | The API returns `200 OK` with the product record needed for the detail screen. | | NOT RUN | | AI Generated | | |

| FR06-API-02 | FR-06 Mobile Product Detail | Mobile | API | Boundary Value Analysis | FR-06 / API 3.2 | Verify the missing-product boundary at and around the lowest id | Backend API is running | `GET /api/products/0`, `GET /api/products/1`, `GET /api/products/2` | 1. Send requests for a below-boundary id, the lowest seeded id, and the next id. 2. Compare the responses. | The API handles the below-boundary or missing id gracefully and returns the expected record for existing ids. | | NOT RUN | | AI Generated | | |

| FR06-API-03 | FR-06 Mobile Product Detail | Mobile | API | Domain Testing | FR-06 / API 4.2 | Verify that a valid cart-add request is accepted | Valid JWT token is available | `POST /api/cart` with a full item body and `quantity: 1` | 1. Send the request directly to the backend with a valid token. 2. Inspect the response. | The API accepts the request and returns a success response. | | NOT RUN | | AI Generated | | |

| FR06-API-04 | FR-06 Mobile Product Detail | Mobile | API | Decision Table / Robustness | FR-06 / API 4.2 | Verify that cart-add requests with missing auth or invalid quantity are rejected | Backend API is running | Missing token, partial JSON body, or `quantity: 0` / `-2` | 1. Send the request without Authorization or

```
with a tampered body. 2. Inspect the response status and body. | The API rejects
the request with a validation or authorization error and does not mutate cart
state. | | NOT RUN | | AI Generated | | |

## Stage F - Self-audit

- Potential duplicate test cases: FR06-UI-04, FR06-UI-05, and FR06-UI-06 all
stress the quantity boundary from different angles, so they are close and should
be reviewed for overlap before execution.
- Missing partitions: I did not add a maximum quantity partition because the
specification does not define one.
- Possibly invented assumptions: the exact wording of the success feedback, the
exact not-found UI, and the category display format may vary by implementation.
- Cases that depend on environment or seed data: FR06-UI-01, FR06-UI-07, FR06-API-
01, and FR06-API-02 depend on seeded product ids and a reachable backend.
- Cases requiring API-level testing: FR06-API-01 through FR06-API-04 must be
executed directly against the backend to bypass the mobile UI.
- Cases requiring a real mobile device: the detail screen layout, touch
interaction, keyboard behavior, and alert visibility are best verified on an iOS
or Android device or emulator.
```

AI Audit Record

Metadata

Field	Value
Feature	FR-07 Shopping Cart
Tool	GitHub Copilot Chat
Timestamp	05/07/2026 13:21

Original Prompt (Verbatim)

```
Apply Test Design to FR-07
```

AI Output (Verbatim)

```
# FR-07 Shopping Cart Test Design

Assumption: Platform is Web (frontend-web) with Backend API coverage, because FR-
07 is implemented in the web client and backed by the server API.

## Stage A - Domain variables

| Variable Type | Variable | Description | Constraint / Relation | Scope |
```

Input	Cart item list	The set of products currently in the cart	Can be empty or populated; same product must map to one logical line item	UI and API
Input	Product identity	Product selected from Home or Product Detail	Must reference an existing product; same product should not create duplicate lines	UI and API
Input	Quantity adjustment	User action to increase or decrease quantity	Quantity changes by one step using plus/minus controls	UI
Input	Delete action	User triggers item removal	Removal must be preceded by a confirmation dialog	UI
Input	Continue shopping action	User clicks the link/button to return to Home	Must navigate back to the home page	UI
Input	Authorization header	JWT header for cart API calls	Required for /api/cart endpoints	API
Input	Cart item payload	JSON body sent to POST /api/cart	Expected fields: id, name, price, quantity	API
System state	Cart ownership	Cart is associated with a user token	Cart data must be isolated per user	API
System state	Empty cart state	Cart contains zero items	Must render empty-state feedback	UI and API
Output	Rendered table	Cart rows and column headings	Must show Sản phẩm, Đơn giá, Số lượng, Thành tiền, Thao tác	UI
Output	Total amount	Sum of all line totals	Must display the correct cart total and the label Tổng cộng	UI
Cross-variable constraint	Same-product merge	Same product added repeatedly	Must increase quantity instead of creating a new row	UI and API
Cross-variable constraint	Deletion confirmation	Delete intent versus confirmed deletion	Only confirmed deletion mutates the cart	UI
Cross-variable constraint	Cart isolation	Two users accessing cart data	One user must not see another user's cart	API

Stage B – Equivalence partitions

Partition ID	Description	Valid or invalid	Representative value	Requirement basis	Risk	Type
UI-V1	Empty cart state renders empty message and shopping link	Valid	0 items	FR-07 empty cart requirement	Medium	Specification-derived
UI-V2	Cart shows one or more distinct products with required columns	Valid	1 product line	FR-07 table columns	High	Specification-derived
UI-V3	Same product added twice should merge into one line with increased quantity	Valid	2 adds of the same product	FR-07 same-product rule	High	Specification-derived
UI-V4	Quantity can be increased or decreased through +/- controls	Valid	Quantity 1 and 2	FR-07 quantity adjustment	High	Specification-derived
UI-I1	Delete action without confirmation does not remove item	Invalid	Cancel in confirmation dialog	FR-07 delete confirmation	High	Specification-derived
UI-V5	Continue shopping returns the user to Home	Valid	Click continue shopping	FR-07 continue shopping requirement	Medium	Specification-derived
API-V1	Authorized GET /api/cart returns current cart for the signed-in user	Valid	Bearer token with empty or populated cart	Cart API auth requirement	High	Specification-derived
API-V2	Authorized POST /api/cart accepts a complete item payload	Valid				

```
{ "id":1, "name":"iPhone 15 Pro Max", "price":30000000, "quantity":1 } | API contract
for cart add | High | Specification-derived |
| API-I1 | Missing or malformed Authorization header | Invalid | No token or wrong
scheme | Cart API auth requirement | High | Specification-derived |
| API-I2 | Partial or malformed cart payload | Invalid | Missing quantity, non-
numeric quantity, or quantity 0 | API schema and quantity behavior | High |
Specification-derived |
| API-V3 | Cart data is isolated per user token | Valid | User A token vs User B
token | Per-user cart scope | High | Specification-derived |
| API-R1 | Very large quantity value for robustness observation | Exploratory |
999999 | No explicit upper bound in spec | Medium | Exploratory |
```

Stage C – Boundary analysis

```
| Boundary | Just below | At boundary | Just above |
| --- | --- | --- | --- |
| Empty versus populated cart | 0 items | 1 item | 2 items |
| Line-item quantity | 0 | 1 | 2 |
| Same-product merge count | 0 repeated adds | 1 repeated add | 2 repeated adds |
| Authorization header presence | Missing header | Valid Bearer token | Malformed
Bearer token |
| Cart line total | 0 đ | Price of 1 item × quantity 1 | Price of 1 item ×
quantity 2 |
```

Stage D – Combination strategy

The test design uses one-factor-at-a-time around a stable baseline so the cart's core behaviors stay traceable without creating a full Cartesian product. For UI coverage, the baseline is a cart with one seeded product and the factors are empty versus populated state, same-product repetition, quantity adjustment, delete confirmation, and navigation back to Home.

For API coverage, the baseline is a valid JWT and a complete item payload. The key factor combinations are token presence, header format, payload completeness, quantity validity, and user isolation. Decision-table thinking is used for the most important interactions: valid token plus valid payload, valid token plus duplicate add, missing or malformed token, and invalid payload values. The exploratory large-quantity case is kept separate from the boundary set so no unstated maximum is implied.

Stage E – Test cases

```
| TC ID | Feature | Platform | Layer (UI / API) | Technique | Requirement ID |
Objective | Preconditions | Test Data | Steps | Expected Result | Actual Result |
Status | Evidence | AI origin | Human revision | Notes |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
- | --- | --- | --- |
| TC-FR07-UI-01 | Shopping Cart | Web | UI | Domain partition | FR-07 | Verify the
empty-cart presentation | Cart is empty | Empty cart | Open /cart | Empty-state
illustration and clear message are shown, along with a continue-shopping link |
| NOT RUN | N/A | AI Generated | Pending | Specification-derived |
| TC-FR07-UI-02 | Shopping Cart | Web | UI | Domain partition | FR-07 | Verify
cart table layout and total label | Cart contains one seeded product | iPhone 15
Pro Max x1 | Open /cart | Table shows Sản phẩm, Đơn giá, Số lượng, Thành tiền,
```

Thao tác; total label reads Tổng cộng | | NOT RUN | N/A | AI Generated | Pending | Specification-derived |

| TC-FR07-UI-03 | Shopping Cart | Web | UI | Equivalence partition | FR-07 | Verify same-product merging in the UI | Cart is empty; product is available | Add product id 1 twice | Add the same product twice, then open cart | One row is shown for that product and quantity becomes 2 | | NOT RUN | N/A | AI Generated | Pending | Same-product rule |

| TC-FR07-UI-04 | Shopping Cart | Web | UI | Boundary value analysis | FR-07 | Verify quantity increment behavior | Cart has one item with quantity 1 | Quantity 1, then click + | Open cart and click + once | Quantity becomes 2 and line total plus cart total increase accordingly | | NOT RUN | N/A | AI Generated | Pending | Boundary at 1 to 2 |

| TC-FR07-UI-05 | Shopping Cart | Web | UI | Boundary value analysis | FR-07 | Verify quantity decrement behavior around the minimum logical quantity | Cart has one item at quantity 1 and one item at quantity 2 for comparison | Quantity 1 and 2, then click - | Open cart and click - on the quantity control | Quantity does not go below 1; when starting at 2 it decreases to 1 | | NOT RUN | N/A | AI Generated | Pending | Minimum logical boundary |

| TC-FR07-UI-06 | Shopping Cart | Web | UI | Decision table | FR-07 | Verify delete confirmation flow | Cart has one item | One cart item | Click Xóa, cancel once, then confirm on the second attempt | Cancel keeps the item; confirm removes the item from the cart | | NOT RUN | N/A | AI Generated | Pending | Confirmation required before mutation |

| TC-FR07-UI-07 | Shopping Cart | Web | UI | State transition | FR-07 | Verify continue-shopping navigation | Any cart state | Any cart contents | Click Tiếp tục mua sắm | User returns to Home | | NOT RUN | N/A | AI Generated | Pending | Navigation requirement |

| TC-FR07-API-01 | Shopping Cart | Web | API | Domain partition | FR-07 | Verify cart retrieval for an authenticated user with an empty cart | Valid JWT for a user with no cart items | Authorization: Bearer valid token | Send GET /api/cart | 200 response and an empty array | | NOT RUN | N/A | AI Generated | Pending | API read path |

| TC-FR07-API-02 | Shopping Cart | Web | API | Equivalence partition | FR-07 | Verify authorized add-to-cart with a complete payload | Valid JWT; product exists | {"id":1,"name":"iPhone 15 Pro Max","price":30000000,"quantity":1} | Send POST /api/cart, then GET /api/cart | Item is stored for that user and returned by GET /api/cart | | NOT RUN | N/A | AI Generated | Pending | Direct backend add |

| TC-FR07-API-03 | Shopping Cart | Web | API | Equivalence partition | FR-07 | Verify duplicate add of the same product is merged logically | Valid JWT; cart starts empty | Same product payload posted twice | Send POST /api/cart twice with the same product, then GET /api/cart | One logical cart item is kept and quantity increases instead of creating a new row | | NOT RUN | N/A | AI Generated | Pending | Critical business rule |

| TC-FR07-API-04 | Shopping Cart | Web | API | Invalid header | FR-07 | Verify missing or malformed Authorization header is rejected | No valid token supplied | Missing header or Authorization: Token abc | Send GET /api/cart or POST /api/cart | Request is rejected with unauthorized or forbidden status and cart data is unchanged | | NOT RUN | N/A | AI Generated | Pending | Header handling |

| TC-FR07-API-05 | Shopping Cart | Web | API | Invalid payload | FR-07 | Verify malformed cart payload is rejected | Valid JWT | Missing quantity, non-numeric quantity, or quantity 0 | Send POST /api/cart with invalid body | Request is rejected and the cart is not mutated | | NOT RUN | N/A | AI Generated | Pending | Bypass UI validation |

| TC-FR07-API-06 | Shopping Cart | Web | API | State isolation | FR-07 | Verify

cart isolation between two users | Two distinct valid JWTs | User A adds item; user B checks cart | User A posts an item; User B sends GET /api/cart | User B does not see User A's items | | NOT RUN | N/A | AI Generated | Pending | Per-user cart scope |
| TC-FR07-API-07 | Shopping Cart | Web | API | Exploratory robustness test | FR-07 | Observe server behavior for an unusually large quantity value | Valid JWT; product exists | {"id":1,"name":"iPhone 15 Pro Max","price":30000000,"quantity":999999} | Send POST /api/cart and inspect the stored value | System behavior is observed and recorded without assuming a maximum that is not specified | | NOT RUN | N/A | AI Generated | Pending | Exploratory, not a boundary |

Stage F – Self-audit

- Potential duplicate test cases: TC-FR07-UI-02 and TC-FR07-UI-04 both observe totals, but one checks layout/labeling and the other checks quantity change; TC-FR07-API-02 and TC-FR07-API-03 both add items, but one checks single insertion and the other checks duplicate handling.
- Missing partitions: no explicit upper bound for cart quantity or cart line count exists in the specification, so no maximum boundary is asserted; mobile-device-specific cases are out of scope because the chosen platform is Web.
- Possibly invented assumptions: the minimum logical quantity of 1 is treated as an implicit boundary for cart adjustment; per-user cart isolation is treated as the intended server-side behavior because the backend stores cart state by user token.
- Cases that depend on environment or seed data: TC-FR07-UI-02, TC-FR07-UI-03, TC-FR07-UI-04, TC-FR07-API-02, TC-FR07-API-03, and TC-FR07-API-06 depend on seeded products and valid user tokens.
- Cases requiring API-level testing: TC-FR07-API-01 through TC-FR07-API-07, especially the invalid header, malformed payload, duplicate-add, and isolation checks.
- Cases requiring a real mobile device: none for this FR-07 report because the selected platform is Web.

AI Audit Record

Metadata

Field Value
--- ---
Feature Rewrite feature
Tool GitHub Copilot Chat
Timestamp 06/07/2026 12:04

Original Prompt (Verbatim)

```
```text
bạn cũng hãy dùng @file:SKILL.md để ghi lại hoạt động rewrite và sửa này lại

AI Output (Verbatim)

```text
Đã chỉnh lại format của [ai/ai_critique_&_audit.md](ai/ai_critique_&_audit.md)
theo skill mới: thêm dòng khai báo `I use AI tools for the following tasks`, giữ
nguyên nguyên văn toàn bộ prompt và output, và bỏ phần Student Review để file khớp
với mục 9 hơn.

Nội dung của các block `Original Prompt` và `AI Output` vẫn được giữ nguyên. Ặp
bạn muốn, tôi có thể tiếp tục áp dụng cùng format này cho các file audit khác nếu
còn.
```

AI Audit Record

Metadata

Field	Value
Feature	FR-01 Account Registration
Tool	GitHub Copilot Chat
Timestamp	06/07/2026 19:34

Original Prompt (Verbatim)

```
Test design for FR-01
```

AI Output (Verbatim)

```
I filled reports/fr01.md with the full FR-01 six-stage test design, scoped to the
web registration page and the backend `/api/register` flow. It includes domain
variables, equivalence partitions, boundary analysis, combination strategy,
traceable UI/API test cases, and a self-audit, all wrapped in the required raw
markdown fence and with `Actual Result` left blank and `Status` set to `NOT RUN`.

I also verified the saved content by reading it back. If you want, I can do the
same for the next feature report.
```